

Retrieval-Augmented Generation: 让 LLM 从闭卷背诵变成开卷查证

图文讲义版：用原 PPT 作为视觉锚点，按“概念故事线 + 直觉解释 + 考试速记”整理。

Closed-book LLM 的知识瓶颈 → Open-Domain QA → Retriever-Reader pipeline → Sparse retrieval → Dense retrieval 与 efficient search → RAG 的概率式证据融合 → Hallucination 与 Attribution → GraphRAG / Agentic RAG

Open Domain QA



Open-Domain Question Answering (ODQA):

We do not assume we are given a passage together with the question

We can only access a large collection of documents (e.g., Wikipedia) — we don't know which document contains the answer, and the goal is to answer any open-domain questions.

Both more challenging and more practical/useful!

这节课一句话

RAG 的核心不是“给 LLM 多塞几段文字”，而是把检索系统的 non-parametric memory 和生成模型的 parametric memory 接起来：先从大规模外部知识里找证据，再让模型基于证据回答、引用和推理。

1. 从 Closed-book LLM 到 Open-Domain QA: 为什么要检索

Open Domain QA



Open-Domain Question Answering (ODQA):

We do not assume we are given a passage together with the question

We can only access a large collection of documents (e.g., Wikipedia) — we don't know which document contains the answer, and the goal is to answer any open-domain questions.

Both more challenging and more practical/useful!

原 PPT 第 4 页

人话直觉

先把场景想清楚：普通 reading comprehension 是老师把文章发给你，然后问问题；Open-Domain QA 是老师只问一句“华沙 1933 年有多少人说 Polish? ”，不给文章，让你自己去整个 Wikipedia 里找证据。LLM closed-book 回答像凭记忆裸考：答得快，但它的记忆可能旧、偏、缺，尤其遇到冷门事实时会开始“自信编故事”。RAG 的动机就是把裸考改成开卷考试。

精确定义 / 机制： Open-Domain Question Answering 不假设 question 已经配好 passage；系统只能访问一个大规模 document collection，并需要先定位可能含答案的 document/passage，再回答。LLM 可以直接用 parametric memory 做 ODQA，但参数知识有更新慢、长尾覆盖差、来源不可追溯、难判断不确定性的限制。RAG 因此引入外部 corpus/index 作为 non-parametric memory，把“模型记住了什么”和“外部证据写了什么”分开。

考试问法：如果考 open-domain QA，先对比 reading comprehension：前者不给 passage，要从大库检索；后者 passage 已给定。若问 RAG 为什么重要，要答它补 LLM 参数记忆的事实性、更新性、长尾知识和可追溯性短板。

2. Retriever-Reader: 先找证据, 再读证据

The Retriever-Reader Framework

Input: a large collection of documents $\mathcal{D} = \{D_1, \dots, D_n\}$ and a question Q

Output: an answer A retrieved from $P_1, \dots, P_k$

Retriever: $\text{retriever}(\mathcal{D}, Q) \rightarrow P_1, \dots, P_k$, where $k \in \mathbb{N}$ is pre-defined (e.g., 100)

Reader: $\text{reader}(Q, \{P_1, \dots, P_k\}) \rightarrow A$, similar to reading comprehension

An early retriever-reader system is **DrQA** [Chen et al., 2017]:

Retriever: a standard, "classic" TF-IDF information retrieval module (fixed)

Reader: a neural reading comprehension model, trained on SQuAD via distant supervision (i.e., by using retrieved paragraphs rather than gold ones)

原 PPT 第 12 页

人话直觉

这套架构像去图书馆回答问题: retriever 是图书管理员, 先从几百万本书里挑出最可能相关的几页; reader/generator 是认真读书的人, 根据这些页抽答案或组织语言。关键是分工: 不要让生成模型一边在全网大海捞针, 一边还要负责写最终答案。

精确定义 / 机制: 给定文档集合 $\mathcal{D} = \{D_1, \dots, D_n\}$ 和问题 Q , retriever 计算 query 与文档或段落的相关性, 返回 top-k passages $P_1, \dots, P_k$; reader 或 generator 接收 Q 与这些 passages, 输出 answer A 。DrQA 是早期代表: 用固定的 TF-IDF 信息检索模块从 Wikipedia 找段落, 再用在 SQuAD 等 reading comprehension 数据上训练的 neural reader 抽取答案。Fusion-in-Decoder 则把多个 passage 分别编码, 在 decoder 中融合证据, 说明生成模型很擅长跨多段证据整合。

$$\text{retriever}(\mathcal{D}, Q) \rightarrow \{P_1, \dots, P_k\}, \quad \text{reader}(Q, \{P_i\}_{i=1}^k) \rightarrow A$$

考试问法: 常见题会让你画 pipeline 或解释模块输入输出: retriever($\mathcal{D}, Q$)->top-k passages; reader/generator($Q, \text{passages}$)->answer。注意 reader-reader 不是 RAG 的全部, 但它是现代 RAG 的直接前身。

3. Sparse Retrieval: 用关键词重叠当路标

Sparse Retrieval

Similarity Score: TF-IDF

Sparse representation over vectors representing vocabulary/terms

$$\begin{aligned} \text{Term frequency} \quad & \text{TF}(w, Q) = \text{count}(w, Q) \\ \text{Inverse Document Frequency} \quad & \text{IDF}(w, \mathcal{D}) = \log \frac{|\mathcal{D}|}{|\text{document freq of } w|} \\ & \mathbf{q}_w = \text{TF}(w, Q) \cdot \text{IDF}(w, \mathcal{D}) \\ & \mathbf{p}_w = \text{TF}(w, P) \cdot \text{IDF}(w, \mathcal{D}) \\ \text{sim}(Q_i, P_j) &= \text{cosine}(\mathbf{q}, \mathbf{p}) \text{ with } \mathbf{q}, \mathbf{p} \in \mathbb{R}^{|\mathcal{V}|} \end{aligned}$$

原 PPT 第 15 页

人话直觉

Sparse retrieval 的想法很朴素：如果问题里有“Warsaw”“1933”“Polish”，那含这些词的段落就像路标更亮。但不是所有词都一样重要——“the”“is”到处都有，几乎没有定位能力；冷门实体词反而像街道门牌号，能迅速缩小范围。

精确定义 / 机制： Sparse retrieval 把 query 和 passage 表示成词表维度上的稀疏向量，典型打分是 TF-IDF 或 BM25。TF 衡量词在当前文本中出现多少，IDF 衡量词在整个文档集中多稀有；二者相乘得到词权重，再用 cosine similarity 比较 query 向量和 passage 向量。它的优点是简单、高效、可解释，对精确实体和术语匹配强；缺点是依赖 lexical overlap，遇到同义改写、隐含语义或跨语言表达时容易漏检。

$$\text{TFIDF}(w, Q) = \text{TF}(w, Q) \cdot \text{IDF}(w, \mathcal{D}), \quad \text{sim}(Q, P) = \cos(\mathbf{q}, \mathbf{p})$$

考试问法： 如果题目问 sparse vs dense，sparse 部分要写：词项向量、词面重叠、TF-IDF/BM25、可解释高效但语义泛化弱。不要只说“传统方法”，要说明它为什么仍是强 baseline。

4. Dense Retrieval: 把问题和段落放进语义空间

Dense Retrieval

Similarity Score:

What if query words and document words do not match?
Dense representation over embeddings

$$\text{sim}(Q_i, P_j) = \mathbf{q}_i^T \mathbf{p}_j \text{ with } \mathbf{q}_i, \mathbf{p}_j \in \mathbb{R}^d$$

$$\mathbf{q}_i = \text{Encode}(Q_i)$$

Entire research on how to improve or
learn the similarity function!

$$\mathbf{p}_j = \text{Encode}(P_j)$$

原 PPT 第 16 页

人话直觉

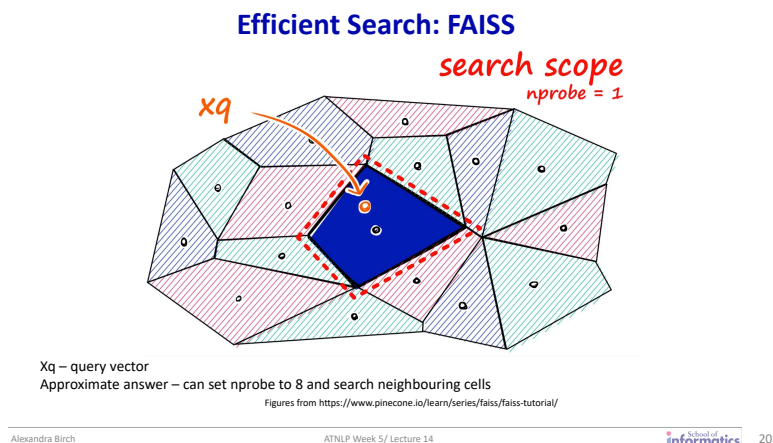
Dense retrieval 解决的是“明明意思一样，词却不一样”的尴尬。用户问“city where Obama was born”，文档写“Barack Obama was born in Hawaii”，如果只靠词面可能错过；dense encoder 试图把语义相近的 query 和 passage 放到向量空间的邻近位置。

精确定义 / 机制：Dense retrieval 用 neural encoder 把 query 编成向量 $\mathbf{q}_i$ ，把 passage 编成向量 $\mathbf{p}_j$ ，并用 dot product 或 cosine similarity 打分。Bi-encoder 分别编码 query 和 passage，passage embeddings 可离线预计算并建立索引，适合第一阶段大规模检索；cross-encoder 把 query 与 document 一起输入模型，token 间可充分交互，通常更准确但计算太贵，更适合 reranking 少量候选。Dense Passage Retrieval 这类方法使 open-domain QA 不再完全依赖词面匹配。

$$\mathbf{q}_i = \text{Encode}(Q_i), \quad \mathbf{p}_j = \text{Encode}(P_j), \quad \text{sim}(Q_i, P_j) = \mathbf{q}_i^T \mathbf{p}_j$$

考试问法：考 dense retrieval 时要抓住 trade-off: bi-encoder 高效、可索引、但交互少；cross-encoder 精细、准确、但不能对全库逐一跑。dense 的优势是 semantic matching，风险是训练分布偏差和实体精确匹配失败。

5. Efficient Search: 为什么向量检索必须近似



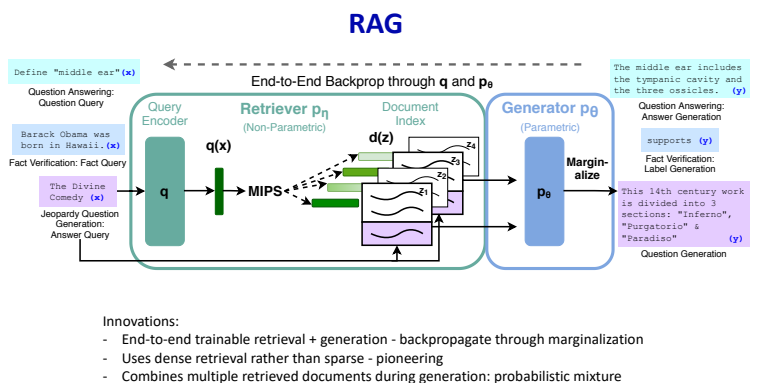
原 PPT 第 20 页

人话直觉

dense retrieval 听起来美，但如果有十亿个 passage embedding，每来一个问题都和十亿个向量点乘一遍，就像为了找一杯咖啡把全城每家店都进一遍。FAISS/IVF 的直觉是先把城市按街区分好：查询向量落在哪几个街区附近，就只搜那些街区。

精确定义 / 机制：Maximum Inner Product Search (MIPS) 要在大量文档向量中找与 query 向量内积最大的候选。精确搜索成本随库规模线性增长，因此实际系统常用 approximate nearest neighbour search。IVF/k-means indexing 先把 embedding space 聚成 centroids，查询时只访问若干相邻 cells；FAISS 是常用的大规模相似度搜索库。它牺牲一点精确性，换取 dense retrieval 在百万到十亿级文档上的可用性。考试问法：如果问为什么 RAG 需要 efficient search，不要泛泛答“为了快”：要点是 top-k retrieval 发生在巨大 datastore 上，dense vectors 不能逐一 exhaustively score；MIPS、IVF、FAISS 让大规模 RAG 可行。

6. 经典 RAG: 把多个检索文档当作 latent evidence



原 PPT 第 22 页

人话直觉

RAG 不是简单复制粘贴 top-1 段落给 LLM。更准确地说，它像侦探同时拿到几份可能相关的证词：每份证词都有可信度，生成答案时要在这些证据之间加权、融合、取舍。slide 22 的 marginalize 就是在说：答案不是只押宝一个文档，而是对多个可能证据求和。

精确定义 / 机制： Lewis et al. 的 RAG 包含 query encoder、dense retriever p_{θ}^{data} 、document index、MIPS 检索和 generator p_{θ}^{heta} 。给定输入 x ，retriever 返回 latent documents z ，generator 基于 x 与 z 生成 y ；训练目标对检索文档边缘化，使 retrieval 和 generation 可以端到端优化。RAG 的贡献包括：dense retrieval 取代纯 sparse 检索、检索与生成联合训练、生成时融合多个 retrieved documents，并把 parametric generator 与 non-parametric index 结合。

$$p(y | x) \approx \sum_{z \in \text{top-}k} p_{\eta}(z | x) p_{\theta}(y | x, z)$$

考试问法：考 RAG 定义时要写清两层 memory：LLM 参数中的 parametric memory 与外部 index/corpus 的 non-parametric memory。考机制时要提 top-k documents、generator conditioned on evidence、marginalization/mixture，而不是只说“先搜再答”。

7. Hallucination 与 Attribution: RAG 让答案可检查，但不是魔法免疫

RAG: Better Attribution

Attribution: Ensure that generated content can be traced to verifiable sources

Direct Generated Attribution	 What type of herbicides are effective against sowthistle? Several herbicides can be effective against sowthistle, depending on the setting and stage of growth^[1]. Here's a list of some common herbicides that can be used ... [1] Seiber et al. xxx, Journal of Environmental Horticulture in 2008. 	Citations can be hallucinated
Post-Retrieval Answering	 What type of herbicides are effective against sowthistle? Clodpyralid and Triclopyr: Selective herbicides containing either clodpyralid or triclopyr can effectively target sowthistle without harming surrounding plants. Some brand examples include Thiallex, Vivendi, Grazon Pro, and Pac-Tor. https://www.wikihow.com/Get-Rid-of-Thistles 	Most reliable but not fool proof. Widely used eg. Perplexity
Post-Generation Attribution	 What type of herbicides are effective against sowthistle? Several herbicides can be effective against sowthistle ... Several herbicides can be...Velocity Herbicide: Particularly in the early growth stages of sowthistle, applying Velocity herbicide post-emergence is a preferred option for control^[1]. https://www.crop.bayer.com/ac/pubs/weeds/sowthistle 	Eg. Bing Chat 2

Li et al. (2023). A Survey of Large Language Models Attribution

Alexandra Birch

ATNLP Week 5/ Lecture 14

School of Informatics

25

原 PPT 第 26 页

人话直觉

RAG 降低 hallucination 的方式不是给模型戴上“不会胡说”的金箍，而是在它旁边放证据纸条。纸条有用，但也可能拿错、读错，或者模型明明看了纸条还自由发挥。所以考试里最容易丢分的说法是“RAG eliminates hallucination”；正确说法是 reduces, not eliminates。

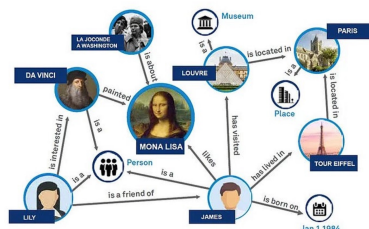
精确定义 / 机制： RAG 对长尾知识特别有效：热门事实可能已被模型参数记住，冷门实体更需要外部检索补充。Attribution 要求生成内容能追溯到可验证来源。slide 26 区分了 direct generated attribution、post-retrieval answering 和 post-generation attribution：直接生成引用可能 hallucinate citation；先检索再回答通常更可靠但仍非完美；先生成再补引用可能只是在事后找支持。Attribution evaluation 还要区分 supported、contradictory 和 extrapolatory。

考试问法：若问 RAG 如何减少 hallucination，答：外部证据约束生成、提升长尾事实覆盖、支持 attribution；同时必须指出 failure modes：retrieval miss、错误证据、context overload、generator 忽略证据、引用

不 faithful。

8. Advanced RAG: 从孤立 chunks 到图结构与 agentic workflow

GraphRAG



Graph RAG Pipeline:

Knowledge Graph: Stores entities and their relationships as nodes and edges.

Graph Retriever: Traverses the graph to find relevant nodes, paths, and multi-hop connections.

LLM: Synthesizes a response using both entities and their relationships, often providing reasoning chains.

Figure from <https://blog.langchain.com/enhancing-rag-based-applications-accuracy-by-constructing-and-leveraging-knowledge-graphs/>

Alexandra Birch

ATNLP Week 5 / Lecture 14

School of Informatics

31

原 PPT 第 31 页

人话直觉

普通 vector RAG 像在资料堆里捞几张相似卡片；GraphRAG 像把卡片上的人物、机构、事件、依赖关系连成地图；Agentic RAG 则像一个会规划的研究助理：先搜一轮，发现证据不够，就改写 query、调用工具、再验证。复杂问题往往不是“一搜一答”能解决的。

精确定义 / 机制： RAG taxonomy 可以从 retrieval granularity、evidence incorporation 和 retrieval frequency 看：检索单位可以是 token、passage、document 或 graph node；证据可以拼接到 input、在中间层融合或影响输出；检索可以一次性、周期性或自适应。GraphRAG 用 knowledge graph 存储 entities 和 relations，通过 graph traversal 找相关 nodes、paths、subgraphs，适合 multi-hop reasoning、关系密集领域和跨文档总结。Agentic RAG 让 agent 动态决定何时检索、检索什么、是否继续验证或调用工具，适合复杂任务但更慢、更贵、错误会累积。

考试问法：如果考 GraphRAG vs RAG，要答：RAG 强在 single-hop/detail-oriented retrieval；GraphRAG 强在 multi-hop、interconnected data、global summarisation。若考 Agentic RAG，要强调 iterative planning/retrieval/verification，而不是一次 top-k 后直接回答。

考试速记版

- Open-domain QA：不给 passage，只给大文档集合，系统必须先检索再回答。
- Closed-book LLM 依赖 parametric memory；RAG 额外接入可更新的 non-parametric memory。
- Retriever-reader pipeline：retriever 选 top-k passages，reader/generator 基于证据输出答案。
- Sparse retrieval 用 TF-IDF/BM25 等词项重叠信号；优点是高效可解释，弱点是语义改写。
- Dense retrieval 用 encoder 把 query/passages 变成向量，以 dot product/cosine 做语义匹配。
- Bi-encoder 可预计算文档向量，适合大规模检索；cross-encoder 交互更强但太贵。
- MIPS、IVF、FAISS 的意义是避免对巨大向量库做 exhaustive scoring。

- 经典 RAG 把 retrieved documents 当作 latent evidence，并对多个文档进行 mixture/marginalization。
- RAG reduces hallucination, not eliminates it: 检索错、证据弱、生成器忽略证据都会失败。
- Attribution 关注答案声明是否被引用证据支持；有引用不等于引用 faithful。
- GraphRAG 适合 multi-hop、关系密集和全局总结；普通 RAG 更适合 single-hop detail retrieval。
- Agentic RAG 用多步规划、检索、验证和工具调用提升复杂任务表现，但成本与错误传播风险更高。

一段稳妥考场回答

Retrieval-Augmented Generation (RAG) 是把生成模型与外部检索系统结合的框架：给定问题后，retriever 先从大规模 corpus/index 中选出 top-k 相关 passages 或 documents，generator/reader 再以这些证据为条件生成答案。它把 LLM 参数中的 parametric memory 与可更新、可检查的 non-parametric memory 结合起来，因此能提升 open-domain QA 的事实性、长尾知识覆盖、知识更新能力和 attribution。实现上，retrieval 可用 sparse 方法如 TF-IDF/BM25，也可用 dense bi-encoder embedding 和 FAISS/MIPS 做大规模语义检索；经典 RAG 还可把多个检索文档作为 latent evidence 进行边缘化融合。但 RAG 不能完全消除 hallucination，因为系统仍可能检索不到关键证据、取回错误证据、让生成器忽略证据或产生不 faithful 的引用。高级方向包括 GraphRAG，它利用实体关系图处理 multi-hop 和全局总结，以及 Agentic RAG，它通过多步规划、改写查询、检索和验证来处理更复杂任务。